

class tableau

```

{ int * t;
  int taille;
  int nbelts;
public: tableau(int*, int, int);
      ~tableau();
      void affiche();
      void saisie();
      int nombre_multi_3();
      float moyenne();
      void supprimer(int);

```

4 (9)

~~tableau A(1000, 20, 15);~~

Après la construction d'un objet

```

taille = 20;
t = new int [taille];
nbelts = 0;

```

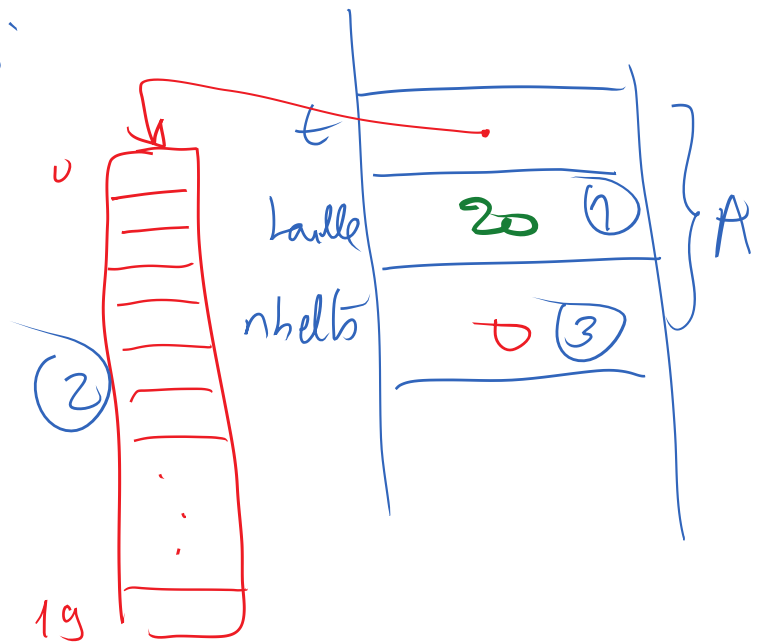


tableau :: tableau (int n = 20)

```

{
    taille = n;
    t = new int [taille];

```

```

    nbelts = 0;
}

```

tableau :: n tableau()

{ delete & ;

}

void tableau :: affiche()

{

for (int i=0; i < nbelts; i++)

{ cout << H[i] << " " ;

}

cout << "\n" ;

}

void tableau :: saisir()

{ ^{do} Cout << "Enter le nombre de elts à saisir <= " << taille << " : ";

cin >> nbelts;

^{while} while (nbelts <= 0 || nbelts > taille);

for (int i=0; i < nbelts; i++)

{ cout << "Enter l'elt numero " << i+1 << " : " ;

cin >> H[i];

}

}

int tableau :: nombre_mult_3()

{ int comp=0;

for (int i=0; i < nbelts; i++)

{ if (H[i] % 3 == 0)

comp++;

}

return comp;

```

4
float tableau; moyenne( )

```

```

{ float sm = 0;

```

```

  for (int i = 0; i < nbelt; i++)

```

```

    { sm += t[i];

```

```

    }

```

```

    return sm / nbelt;

```

```

}

```

tableau A(10);

A.saisie();

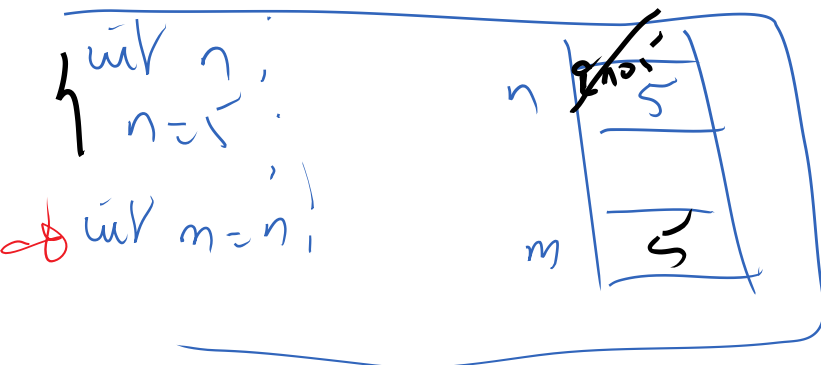
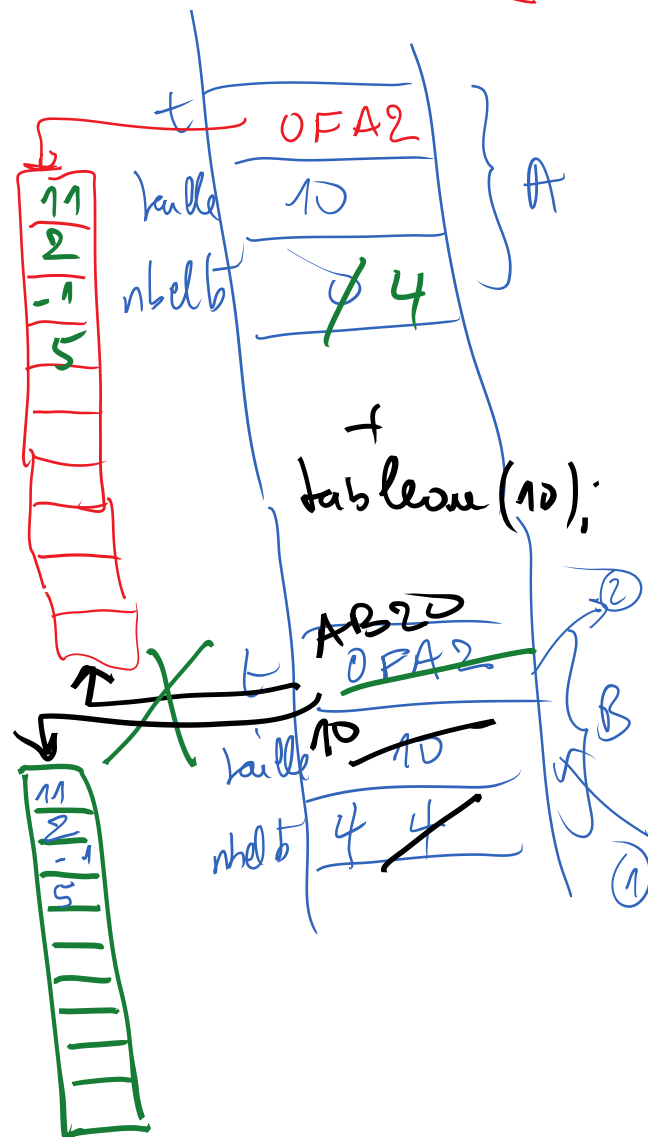


tableau B = A;

~~to. tableau(A)?~~



B.taille = A.taille; ①

B.t = new int[B.taille]; ②

B.nbelt = A.nbelt; ③

for (int i = 0; i < B.nbelt; i++)

④

```

    } B.t[i] = A.t[i];
    }

```

```

tableau; tableau (const tableau &v)

```

{ Constructeur
de copie ou
re copie

```

{   taille = v.taille;
    t = new int [taille];
    nbelt = v.nbelt;
    for (int i = 0; i < nbelt; i++)
    {   t[i] = v.t[i];
    }
}

```

tableau A(5); $\Rightarrow$ tableau(5)

tableau B(A); $\Rightarrow$ tableau(A)

tableau B = A;

class nom classe

{ ① ~ ;

② printer ;

public: nom classe (①) ;

void affiche() ;

~ nom classe () ;

nom classe (const nom classe &) ;

⋮

class pile

{ int * t ;

int taille ;

int sommet ;

public: pile(int) ;

pile (const pile &) ;

~ pile () ;

~~void affiche () ;~~

void empiler (int) ;

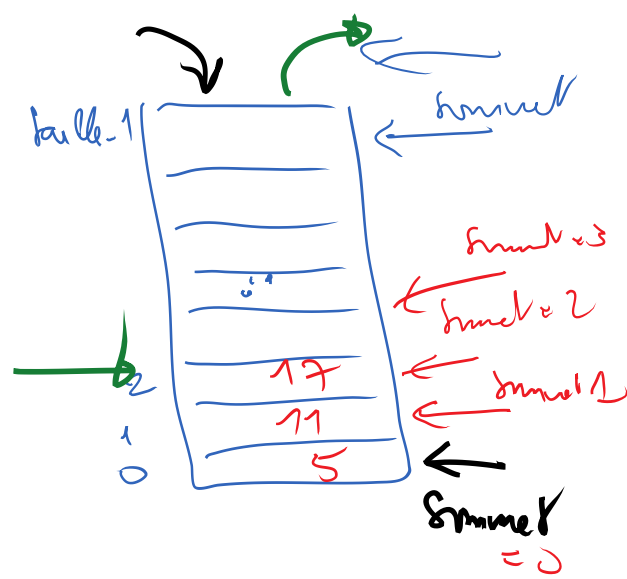
int pleins () ;

int depiler () ;

int vide () ;

void RAZ () ;

4 [9]



```

pile :: pile (int n > 0)
{
    taille = n;
    t = new int [taille];
    summed = 0;
}

```

```

pile :: pile (const pile & v)
{
    taille = v.taille;
    t = new int [taille];
    summed = v.summed;
    for (int i = 0; i < summed; i++)
    {
        t[i] = v.t[i];
    }
}

```

```

pile::~~pile()
{
    delete t;
}

```

```
int pile :: pleine()
{ if (sommet == taille)
    return 1;
  else
    return 0;
}
```

sommet == taille

"
sommet <= taille

} return (sommet == taille);

81